Managing and working with application resources in a hierarchy

Tanusree Nath
PhD Research Scholar
Department of Computer Science, BHU

Google scholar: https://scholar.google.com/citations?user=h80-tYYAAAAJ&hl=en
LinkedIn: https://www.linkedin.com/in/tanusree-nath-5447291a7/

Android Resources

- Resources are the **non-code** components of your app. By keeping them separate from your Kotlin logic, you make your app easier to maintain and translate.
- All resources live in the res/ directory of your project.

Resource Type	Folder Location	Description
Layouts	`res/layout/`	XML files that define your UI structure.
Strings	`res/values/`	Text content (labels, hints, messages).
Images	`res/drawable/`	Bitmaps (PNG/JPG) or Vector graphics.
Colors	`res/values/`	Hex codes for your UI theme.

The Power of "Externalizing"

Why bother putting things in separate folders?

- **Reusability:** Use the same string or color in ten different screens. Change it once in the XML, and it updates everywhere.
- **Localization (L10n):** Easily provide translations. You can have a strings.xml for English and another for Spanish; Android picks the right one based on the user's phone settings.
- **Device Adaptability:** You can provide a high-resolution image for tablets and a lower-resolution one for small phones without changing a single line of code.

Understanding the Resource Directory Hierarchy

- The res/ directory is organized into specific sub-folders based on the **type** of content they hold. This hierarchy is strict; Android expects certain files to be in certain places to find them.

```
res/
  ├── layout/
  ├── drawable/
  ├── mipmap/
  ├── values/
  ├── menu/
  └── raw/
```

The Standard Folder Breakdown

Folder	What goes inside?	Common File Types
layout/	The "blueprints" for your screens. Defines where buttons and text go.	activity_main.xml
drawable/	Visual assets like icons, shapes, and background images.	.png, .webp, .xml (vectors)
mipmap/	Specifically for **App Launcher Icons** (the icon on the home screen).	ic_launcher.png
values/	Simple data like text strings, color hex codes, and font sizes.	strings.xml, colors.xml
menu/	Definitions for navigation drawers or top-bar options menus.	toolbar_menu.xml
raw/	Files that shouldn't be compressed, like audio or video clips.	.mp3, .mp4, .json

How the Hierarchy Works

1. **Fixed Naming:** You cannot rename these folders. If you rename layout to my_screens, Android won't know where to look, and your app will crash.
2. **The R Class Bridge:** Every time you add a file to these folders (e.g., res/drawable/logo.png), Android automatically creates an ID for it: R.drawable.logo. This is how you "call" the file in your code.
3. **No Sub-folders:** You cannot create folders *inside* these folders (e.g., layout/home/ is not allowed). You must keep all files directly inside the main category folder.

Resource Qualifiers

- Resource qualifiers make Android apps look professional on millions of different devices. Instead of writing logic like if (screen == tablet), you simply create a folder with a specific suffix, and **Android does the heavy lifting for you.**
- A qualifier is a string added to a resource directory name using a **hyphen**. The syntax is: <resource_name>-<qualifier>

Resource qualifiers - How they work

Some examples:

Folder Name	Qualifier Type	When is it used?
layout-land	**Orientation**	When the device is rotated horizontally.
layout-sw600dp	**Screen Size**	Specifically for tablets (7" and up).
values-night	**UI Mode**	When "Dark Mode" is active.
values-en-rGB	**Region**	When the language is English (UK specifically).
drawable-hdpi	**Density**	For screens with high pixel density.

Resource Qualifiers - The "Selection Process" (Rule of Best Match)

When your app runs, Android follows a specific "filtering" process to find the right resource:

1. **Elimination:** It looks at all available folders. If the device is in **Portrait** mode, it immediately ignores any folder with -land.
2. **Priority:** Android follows a strict hierarchy of importance (e.g., Language has higher priority than Orientation).
3. **The "Default" Fallback:** If no qualified folder matches the device state (e.g., you don't have a -night folder), Android pulls the resource from the **default** folder (the one without a hyphen, like res/layout/).

Resource qualifiers - Example Scenario

Imagine you are building a Login screen:

- In **res/layout/activity_login.xml**: You put your buttons in a vertical column (Default).
- In **res/layout-land/activity_login.xml**: You put the buttons side-by-side so they fit on a wide, short screen.

The Code: You still just call **setContentView(R.layout.activity_login).** Android handles switching the actual layout file when the user rotates their phone!

Types of resource qualifiers

To master Android development, you need to know how to target specific device traits. Here are the "Big Three" categories of qualifiers you will use in almost every project:

- Device Configuration (Hardware)
- Localization (L10n)
- UI Mode (Themes)

Types of resource qualifiers – Device Config

- These qualifiers ensure your UI looks good regardless of the physical screen it’s running on.
- **Orientation (-land / -port):**
  - layout-land/: Used when the phone is sideways (landscape).
  - layout-port/: Used when the phone is upright.
- **Smallest Width (-sw<N>dp):**
  - layout-sw600dp/: This is the standard for **7-inch tablets**. "sw" stands for Smallest Width. If a device has at least 600dp of width, Android picks this.
- **Screen Density (-hdpi, -xhdpi, etc.):**
  - Android devices have different pixel densities (some have tiny pixels, some have huge ones). You provide different image sizes so they don't look blurry.
  - drawable-mdpi/ (1x baseline)
  - drawable-hdpi/ (1.5x)
  - drawable-xhdpi/ (2.0x)

Types of resource qualifiers – Localization (L10n)

- This is how you make your app global. You keep your strings.xml file name the same, but put it in different folders.
- **Language (-en, -hi, -fr):**
  - values-en/strings.xml → "Hello"
  - values-hi/strings.xml → "नमस्ते"
- **Region (-rUS, -rIN):**
  - You can even target specific regions, like values-en-rGB/ for British English (e.g., "Colour" instead of "Color").

Types of resource qualifiers – UI Mode

- Modern apps must support the user's system preferences, specifically their eyes at night!
- **Night Mode (-night):**
  - values-night/colors.xml: You define darker background colors and lighter text colors here.
  - When the user toggles **Dark Theme** in their system settings, Android instantly switches from values/ to values-night/.

Resource Resolution Mechanism

The "Resource Resolution Mechanism" is the set of rules Android follows to decide exactly which file to display. It is a process of **elimination** and **prioritization**.

- **The Step-by-Step Search**
- When the app requests a resource (like R.string.welcome), Android performs these three steps in order:
- **Look for an Exact Match:** It checks if there is a folder that matches *all* current device configurations (e.g., specific language + specific orientation).
- **Look for the Closest Match:** If an exact match doesn't exist, it drops the least important qualifier and looks again.
- **Default Fallback:** If no qualified folders match, it goes to the "base" folder (the one with no hyphens). **This is why you must always have a default version of every resource.**

Resource Resolution Mechanism – Real-World Example

Scenario: A user is using your app on a phone set to **Hindi** and held in **Landscape** mode.

Priority	Android Searches For...	Result
1st Choice	values-hi-land/	**Exact Match:** Perfect fit.
2nd Choice	values-hi/	**Partial Match:** Correct language, ignores orientation.
3rd Choice	values/	**Default:** Standard English/System default.

What happens if values-hi-land is missing? Android moves to values-hi. If that is also missing, it shows the text in the default values/ folder. This ensures the app doesn't crash, even if you haven't translated everything yet!

The Precedence Rule (Priority List)

Android doesn't treat all qualifiers equally. Some have higher "weight" than others. If two folders could potentially match, Android uses the one with the higher-priority qualifier.

Top Priority Qualifiers –Highest to Lowest (Simplified):

- **MCC/MNC** (Mobile Country Code / Mobile Network Code)
- **Language/Region** (-en, -hi) — *Very High Priority*
- **Layout Direction** (-ldrtl)
- **Smallest Width** (-sw600dp)
- **Orientation** (-land, -port)
- **Screen Density** (-hdpi)

res/values/ Resource Hierarchy

The values/ directory holds **non-layout** resources like strings, colors, dimensions, styles, and arrays. Unlike drawables or layouts, values resources are **not tied to screen density or orientation** in the same way, but they still follow the **qualifier precedence rules**.

Inside the res/values/ folder, you don't just put one giant file. Android encourages you to group simple data types into specific XML files. This keeps your project organized and makes it easy to swap values based on the device configuration.

Common Files in res/values/:

File Name	Purpose	Example Entry
strings.xml	All UI text (labels, buttons, hints).	<string name="login">Login</string>
colors.xml	Hex codes for your brand colors.	<color name="primary">#FF6200EE</color>
dimens.xml	Sizes, margins, and padding (in dp or sp).	<dimen name="padding_small">8dp</dimen>
styles.xml	Reusable UI looks (themes and font styles).	<style name="BigText">...</style>

The "Same Name, Different Value" Concept

This is the most important part of resource management. You use the **exact same name** (ID) for a resource across different folders. Your code only ever calls the name; Android decides which version of the "value" to plug in.

Example: Localization

If your code asks for R.string.title, Android looks at the language settings:

1. Default Folder (res/values/strings.xml)

```
<string name="title">Hello</string>
```

2. Hindi Folder (res/values-hi/strings.xml)

```
<string name="title">नमस्ते</string>
```

Referencing Resources

Now that you’ve organized your resources into folders, you need to know how to "pull" them into your app. There are two main ways to do this: within your **UI layout (XML)** and within your **logic (Kotlin)**.

Referencing resources – Inside XML

When you are building your UI in an XML layout file, you reference resources using the @ symbol. This tells Android, *"Don't use this literal text; go look for a resource with this name."*

Syntax: @[package:]type/name

- **String:** android:text="@string/title"
- **Color:** android:background="@color/primary_blue"
- **Dimension:** android:padding="@dimen/main_margin"
- **Drawable:** android:src="@drawable/logo"

Referencing resources – Inside Kotlin (The R Class)

In your code, you use the **R class**. The R class is an automatically generated index that maps every resource in your res/ folder to a unique integer ID.

Syntax: R.type.name

- **To get a String:** val myTitle = getString(R.string.title)
- **To set an Image:** imageView.setImageResource(R.drawable.logo)
- **To get a Color:** val color = ContextCompat.getColor(this, R.color.primary_blue)

Wait, what is R? > Think of R as a **table of contents** for your res/ folder. Instead of the computer searching through folders for "title.xml," it looks up the ID number (like 0x7f04001) assigned to that name in the R class. It’s much faster!

res/drawable/ Resource Hierarchy

One of the biggest challenges in Android is the sheer variety of screen sizes and "pixel densities." To keep your images looking sharp (not blurry or pixelated), Android uses a density-based hierarchy.

Instead of creating an image for every single phone model, Android groups screens into "buckets" based on how many pixels are packed into one inch (DPI).

Folder Name	Density Level	Scale Factor	Best for...
`drawable-mdpi/`	Medium (160 dpi)	**1.0x**	The Baseline (Old/Budget phones)
`drawable-hdpi/`	High (240 dpi)	**1.5x**	Older mid-range phones
`drawable-xhdpi/`	Extra High (320 dpi)	**2.0x**	Standard modern smartphones
`drawable-xxhdpi/`	Extra Extra High	**3.0x**	High-end flagship phones

How Android Picks the Resolution

- When you put an image named logo.png in all these folders, Android performs **Automatic Selection**:
- **Check Hardware:** Android detects the device's screen density (e.g., a Google Pixel might be xxhdpi).
- **Match Folder:** It looks for res/drawable-xxhdpi/logo.png.
- **The "Upscale/Downscale" Backup:** * If you *only* provide an mdpi image, Android will "stretch" it to fit a high-res screen, making it look **blurry**.
  - If you *only* provide an xxhdpi image, Android will "shrink" it for a low-res screen, which uses up **too much RAM**.

res/layout Variants (Responsive UI)

In the world of Android, "Responsive UI" means your app shouldn't just look like a stretched-out phone app when opened on a tablet. You achieve this by creating **Layout Variants**.

Just like with Strings and Drawables, you keep the **file name exactly the same**, but you change the **internal XML design** to fit the screen.

The Layout Hierarchy

Folder	Target Device / State	Design Strategy
layout/	Standard Phones (Portrait)	Vertical stacks, single-column lists.
layout-land/	Any device in Landscape	Side-by-side views, wider buttons.
layout-sw600dp/	7" Tablets and larger	"Master-Detail" flows (list on left, content on right).

Best Practices for Resource Management

"Golden Rules" that professional Android developers follow. These ensure your app is stable, efficient, and easy to maintain.

1. Always Provide Default Resources

The Rule: Every resource must exist in a "default" folder (e.g., values/, layout/, drawable/).

Why? If a user is on a device with a configuration you didn't plan for (like a new language), the app will look for the default. If the default is missing, the app will **crash**.

Analogy: The default folder is your "Safety Net."

Best Practices for Resource Management (contd.)

2. Avoid Duplication (The DRY Principle)

- **The Rule:** "Don't Repeat Yourself." If a layout is the same for both portrait and landscape, don't create two files. Just keep one in the default layout/ folder.
- **Why?** If you have two identical files and you need to change a button color, you'll have to remember to change it in both places. It's better to have one "source of truth."

3. Use dp and sp Units

- **The Rule:** Never use px (pixels).
- **dp (Density-independent Pixels):** Use for layouts, margins, and sizes. It ensures a button looks the same physical size on a tiny phone and a giant tablet.
- **sp (Scale-independent Pixels):** Use for **text**. It behaves like dp, but also scales based on the user's "Font Size" settings in their phone's accessibility menu.

Best Practices for Resource Management (contd.)

4. Use Qualifiers Instead of Runtime Checks

- **The Rule:** Let the folder system do the work.
- **Bad Practice:** Writing code like if (screenSize > 600) { loadTabletUI() }.
- **Best Practice:** Create a layout-sw600dp/ folder.
- **Why?** It keeps your Kotlin code clean and focused on logic, while the XML handles the presentation.

5. Keep Naming Consistent

- **The Rule:** Use the exact same ID (android:id) for the same component across different layout variants.
- **Why?** Your code looks for a specific ID to attach logic (like a click listener). If your phone layout uses btn_submit but your tablet layout uses submit_button, the app won't find the view and will throw a NullPointerException.

Common Mistakes

Even experienced developers fall into these traps. Understanding these mistakes will help you debug your apps much faster.

- **1. Missing Default Resources (The #1 Cause of Crashes)**
- **The Mistake:** Creating a resource in a qualified folder (like values-hi/strings.xml) but forgetting to put it in the default folder (values/strings.xml).
- **The Result:** If a user whose phone is set to English opens the app, Android looks for the string in values-en/ (doesn't exist) and then values/. If it’s not there either, the app **crashes immediately** with a ResourceNotFoundException.
- **The Fix:** Always build your "default" version first, then add translations or variations later.

Common Mistakes (contd)

2. Wrong Qualifier Usage & Order

- **The Mistake:** Using the wrong folder name or putting qualifiers in the wrong order.
- **Naming:** Using uppercase (e.g., layout-Land/) or using underscores instead of hyphens (layout_land).
- **Order:** Android has a strict "Priority List." For example, you must put the language before the orientation (follow qualifier precedence).
  - layout-land-en/ (Wrong order)
  - layout-en-land/ (Correct order)
- **The Fix:** If you right-click the res folder and use **New > Android Resource File**, Android Studio will automatically format the folder name and qualifier order for you.

Common Mistakes (contd)

3. Hardcoding Values

- **The Mistake:** Writing text or colors directly in your XML or Kotlin code.
- **Example (Bad):** android:text="Login Now"
- **Example (Good):** android:text="@string/btn_login"
- **Why it's a mistake:** Hardcoding makes your app "locked." You can't translate it, you can't change the theme (Dark Mode), and if you use that same "Login" text in five places, you have to change it five times instead of once.

Common Mistakes (contd)

4. Inconsistent IDs in Layout Variants

- **The Mistake:** Changing the android:id of a view in a variant layout.
- **Scenario:** You have a button @+id/submit_btn in layout/activity_main.xml. In layout-land/activity_main.xml, you rename it to @+id/btn_send.
- **The Result:** When the user rotates their phone, your Kotlin code (which is looking for submit_btn) will return null. This leads to the dreaded **NullPointerException**.
- **The Fix:** Ensure all views that your code interacts with have **identical IDs** across every single layout folder.

Common Mistakes (contd)

5. Using the Wrong Directory Type

- **The Mistake:** Putting images in the layout/ folder or XML files in the mipmap/ folder.
- **The Result:** The project will fail to compile. Android is very strict about what goes where.
- **The Fix:** Remember the hierarchy:
  - **XML UI** → layout/
  - **Icons/Images** → drawable/
  - **App Launcher Icons** → mipmap/

Thank You